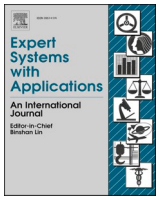
Expert Systems With Applications 238 (2024) 122192

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



Class-imbalanced time series anomaly detection method based on cost-sensitive hybrid network

Xiaofeng Wang [a,*], Ying Zhang [a], Ningning Bai [a], Qinhua Yu [a], Qin Wang [a]

Xi'an University of Technology, Xi'an, China

ARTICLE INFO

Keywords:
Class-imbalanced time series
Anomaly detection
Convolutional neural network
Gated recurrent unit

ABSTRACT

The time series is a structured data form. In practical engineering, most of the time series is in the form of class-imbalance, in which, abnormal data is rare or underrepresented. In this study, we present a cost sensitive hybrid network (CSHN) model to detect data anomaly in class-imbalanced time series. The proposed model adopts a two-flow structure that consist of a Convolutional Neural Network (CNN) and a Gated Recurrent Unit (GRU) network and combine with a cost-sensitive loss function. This model integrates the advantages of CNN and GRU. The former is provided with strong learning ability for local state features and the latter can extract the sequential features and force the decision logic of the network to pay more attention to the long-term dependence of the time series. The cost-sensitive loss function is used to solve the problem that the detection accuracy of the minority class will be inaccuracy caused by skewed data distribution. The simulation experiments are conducted on the speed and temperature datasets that come from real-world measurement and control engineering, as well as the UCR datasets. Compared to using the cross-entropy loss function, the using of the cost-sensitive loss function can improve *G*-means and *F*-measure by 3 %~5% and 5 %~9%, respectively, on the speed and temperature datasets. Experimental results demonstrate that our method is efficient for the class-imbalanced data sets, and the numerical evaluation indexes are superior to that of the comparison methods.

1. Introduction

Time series is an important data types, which are ubiquity in many domains such as medicine (Liang et al., 2020), industry (Mehdiyev et al., 2017), meteorology, finance, aerospace, and so on. Especially, in the engineering of aerospace measurement & control, there are large amount of time series data. Whether the telemetry data is normal or not, directly reflects the running status of spacecraft. Therefore, for time series data, anomaly detection is a very important task in academia and industry.

Time series anomaly detection is essentially a problem of time series classification. It is considering as one of 10 most challenging problems of data mining domain, and has been widely concerned in recent years (Yang and Wu, 2006). Back in the 1990s, there were time series classification methods that combined manual feature extraction with expert knowledge. Owing to the uncertainty of manual feature extraction, the classification performance of this kind of methods is very unstable. In recent years, with the growing demand for big data mining, traditional data analysis methods are far from meeting the requirements of industry and academia. In the beginning of 21st century, data-driven machine learning approach was presented to solve the issue of massive data analysis. Especially, deep learning has been successfully applied in image recognition (Shen et al., 2018), machine translation (Yang et al., 2019) in recent years. Many scholars have devoted themselves to use deep learning for time series classification, and abundant research achievements have presented.

However, in practical engineering, most time series data are in the form of class-imbalance (He, 2011). That is, the number of normal samples is much more than that of abnormal sample, thus the data set is presence of skewed class distributions and underrepresented for minority class. For example, the telemetry data obtained from the aerospace measurement and control engineering, the normal samples are overwhelming. On this occasion, the classification accuracy of the anomaly (minority) class will be seriously affected in the learning-based classification method. The reason is that standard learning-based classification model usually tends to bias toward the majority class because of focus on maximizing the overall classification accuracy, thus, the correctly detecting and identifying anomaly samples are more difficult.

* Corresponding author at: Xi'an University of Technology, Xi'an 710048, China.
E-mail addresses: xfwang66@sina.com.cn (X. Wang), zhangying_work@163.com (Y. Zhang), bnn@stu.xaut.edu.cn (N. Bai), ciciqinhuayu@sina.com (Q. Yu), q_wang96@126.com (Q. Wang).

https://doi.org/10.1016/j.eswa.2023.122192
Received 26 September 2022; Received in revised form 6 September 2023; Accepted 14 October 2023
Available online 18 October 2023

However, in practical application, the anomaly (minority) class samples are more worthy for attention. E.g., the data information from aircraft accidents is very scarce, but it is far more important in predicting unknown failures than the vast amount of data on normal flights. Therefore, it is great important that the research for time series classification method needs to take the problem of class-imbalance into consideration.

The classification task for imbalanced time-series is more challenging than that of common datasets due to the high-dimensional nature and strong temporal correlations between adjacent time points. In this study, aiming at the problem that the detection accuracy of the minority class will be inaccurate due to skew data distribution, we propose a cost-sensitive hybrid network model for anomaly detection in imbalanced time series data. The main contributions are as follows.

(1) We propose a cost-sensitive hybrid network (CSHN) model for class-imbalanced time series classification. In the proposed model, we employ a cost-sensitive loss function to integrate the Convolutional Neural Network (CNN) and Gated Recurrent Unit (GRU) network, harnessing the complementary strengths of both architectures. CNN excels in local feature learning while GRU demonstrates superior sequence feature learning capabilities.
(2) We employ a cost-sensitive loss function in the CSHN model to address the issue of inadequate feature learning for minority class. The core idea revolves around utilizing distinct penalty factors to penalize misclassifications made by the network model. In cases where minority class samples are misclassified, we apply a larger penalty factor to amplify the overall loss. Conversely, when majority class samples are misclassified, we utilize a smaller penalty factor to regulate the increase in total loss.

2. Related work

The task of time series anomaly detection is essentially a problem of time series classification. Existing techniques mainly applicable to standard data sets. Many studies that aimed to imbalanced time series classification toward to lift the significance of the minority classes. We investigated typical methods and analyzed as follows.

A. Time Series Classification Method for Standard Data Set.

1) Distance-based time series classification methods.

The idea behind this type of methods is to employ a distance function for quantifying the similarity between two time series, followed by utilizing established classifiers such as K-Nearest Neighbor (K-NN) or support vector machine (SVM) (Jeong and Jayaraman, 2015) for conducting classification. In these methods, Euclidean distance (ED) (Ding et al., 2008) has been widely used as distance function. Because this kind of methods are based on point-to-point calculation, they are not suitable for two unequal time series. In addition, this type of methods is sensitive to missing values and noise of time dimension. To overcome these shortcomings, the Dynamic Time Warping (DTW) distance function (Li, 2015b) was proposed. The DTW algorithm starts by creating a distance matrix where each element is a pairwise distance corresponding to two points in two time series. Then, in the possible paths of the matrix, the path with the smallest sum of DTW distances is found to judge the similarity of two time series. Although DTW method has a significant improvement in accuracy, it has a large quadratic spatial and temporal complexity.

2) Features-based time series classification methods.

Most of the original time series data are characterized by high dimensionality, which leads to a significant computational cost when classifying such data. Therefore, it is imperative to employ appropriate feature extraction methods in order to extract meaningful features from these high-dimensional time series data sets. Some commonly used techniques include PCA (Principal Component Analysis)-based method (Li, 2015a), SVD (Singular Value Decomposition)-based method (Groutage and Bennink, 2000), and sparse-coding-based method (Bengio et al., 2013). Ye et al. (Ye and Keogh, 2011) reported a method based on Shapelets feature representation. Shapelets is a sub-sequence that can represent the category of a time series. In (Ye and Keogh, 2011), authors applied the method of exhaustive search to find suitable Shapelets. To improve computing effect, a Fast Shapelets (FS) algorithm (Rakthanmanon and Keogh, 2013) was presented. In (Rakthanmanon et al., 2013), authors converted original high-dimensional time series datasets into discrete low-dimensional small datasets by using the method of symbolic aggregation approximation, then searched Shapelets quickly on small datasets.

3) Ensemble-based time series classification methods.

The main concept behind these methods is the integration of diverse classifiers for classification purposes. In comparison to basic classifiers, ensemble classifiers typically exhibit significantly enhanced performance in classification tasks. Random Forests (RF) (Hao et al., 2015) and Adaboost (Rajesh and Dhuli, 2018) are representative ensemble learning algorithms that utilize decision tree classifiers. RF employs bagging techniques while Adaboost is based on boosting techniques; both approaches combine numerous decision trees and employ voting principles for classification purposes. Lines et al. (Lines and Bagnall, 2015) introduced an Elastic Ensemble (PROP) method consisting of 11 classifiers, whereas Work (Bagnall et al., 2015) presented an ensemble classifier comprising 35 distinct classifiers.

4) Deep learning-based time series classification method.

The aforementioned approaches are all rooted in traditional machine learning techniques. These methods necessitate extensive data preprocessing and well-crafted sup. plementary methodologies for feature extraction. Evidently, the process of feature extraction is both time-consuming and labor-intensive. With the advancement of artificial intelligence, deep learning has emerged as a successful paradigm for analyzing time series data. Zheng et al. (Zheng et al., 2016) presented a method that is based on a multi-channel convolutional neural network (MC-CNN). In this work, authors used three channels to learn the features of time series, then these features were pooled into the SoftMax layer for final classification. This method can be used for multivariate time series classification. Cui et al. (Cui et al., 2016) proposed a multi-scale Convolutional Neural Networks (MCNN) model for time series classification. The advantage of the MCNN is setting the multiple branch translation layers that conduct data preprocessing in the time domains and frequency domains. This model can be used to automatically extract features of different scales by establishing multiple local convolution layers. Through performance evaluation on published benchmark UCR datasets (Chen et al., 2015) that contains 44 benchmark datasets, the MCNN model has good results on only 10 datasets. Both MC-CNN model and MCNN model need heavy preprocessing, and manually set learning rate, therefore, the cost of the hyper-parameter settings is heavy. To improve computational efficiency, works (Wang et al., 2017b) and (Karim et al., 2017) proposed to use the global average pooling layer instead of the fully connected layer to reduce the network parameters. In (Wang et al., 2017), authors tested deep multi-layer perception (MLP), fully convolutional networks (FCN) and the residual networks (ResNet) for time series classification. Through performance evaluation on 44 benchmark datasets, these models show good experimental results on 18 datasets. To further enhance network performance, (Karim et al., 2017) proposed two integrated network models: Long Short Term Memory Fully Convolutional Networks (LSTM_FCN) and Attention Long Short Term Fully Convolutional Networks (ALSTM_FCN). The ALSTM_FCN model was evaluated on 85 benchmark datasets, demonstrating promising experimental results across 51 datasets. However, it should be

noted that the ALSTM-FCN model requires learning a larger number of parameters during the training stage, resulting in lower learning efficiency.

B. Class-imbalanced Time Series Classification Method.

The techniques for handling class-imbalanced data include three commonly used methods (He and Garcia, 2009): data-based methods, algorithm-based methods, kernel-based methods, and active learning methods. In this section, we provide a brief overview of the traditional and prominent works in class-imbalanced data learning in recent years.

Data-based methods manipulate the data distribution through data processing techniques. In these approaches, sampling is the most commonly employed method, which includes under-sampling for the majority class and over-sampling for the minority class. The former aims to balance datasets by randomly removing samples from the majority class; however, it has a drawback of potential loss of valuable information (Liu et al., 2008). On the other hand, the latter adjusts data distribution by randomly duplicating or inserting samples from the minority class. This approach typically results in over-fitting in cases where there is a serious class-imbalance. To overcome these disadvantages, Chawla et al. (Chawla et al., 2002) proposed the synthetic minority over-sampling technique (SMOTE) that increases the number of minority class samples by interpolating.

Algorithm-based methods do not change the distribution form of the data set, and they utilize algorithms to reduce the impact on classification results caused by imbalanced data distribution. These algorithms contain threshold moving method and cost sensitive method (Zhou and Liu, 2005). For the former, the decision threshold of classifier is adjusted by manually setting. Owing to the need to find appropriate threshold, it is difficult to detect amphibious samples. Cost sensitive learning methods focus on reinforcing the sensitivity of classifiers to under-represented classes. It uses cost parameters with matrix form that contains different misclassification costs associated with each class sample; or use weight coefficients as cost parameters, by this way, the class-wise costs can be incorporate into the objective function of classifier during the training process, then adjusts the classification performance of classifier by using different punishments to misclassification. These methods is provided with satisfactory performance on binary data (Ruwani et al., 2021).

With the successful application of deep learning in visual classification, class-imbalanced data classification methods that are based on deep learning have also received great attention. Among recent contributions, Lin et al. (Lin et al., 2017) reported a loss function called focal loss (FL) for object detection. In (Lin et al., 2017), authors promote the detection performance for harder samples by reducing the loss weight assigned to well-classified instances. Khan et al. (Khan et al., 2017) presented a cost sensitive deep neural network. This network can extract features from both majority and minority classes. Ren et al. (Ren et al., 2018) reported a *meta*-learning-based approach to training biases and approximately triples the training time compared to conventional methods. Zhang et al. (Zhang et al., 2018) proposed an evolutionary cost-sensitive deep belief network (ECS-DBN) for class-imbalanced data classification. This method is very expensive because the cost of class-dependent misclassification is first optimized by an adaptive differential evolution algorithm (Ruwani et al., 2021).

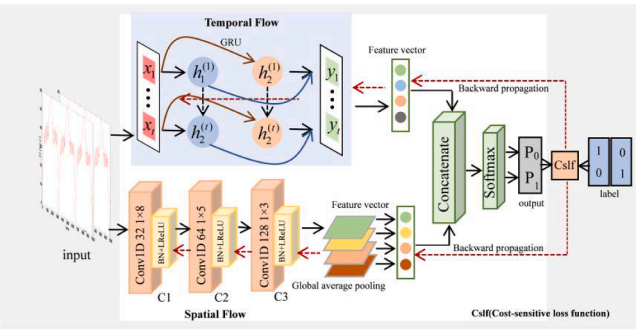


Fig. 1. The architecture of the proposed CSHN model.

Most learning-based imbalanced data learning methods adjust classifier preferences by designing appropriate loss functions. Dong et al. (Dong et al., 2018) proposed a model for imbalance data learning. Their network architecture contains a class rectification loss (CRL) function, and authors adopt batch-wise hard sample mining in majority (frequently sampled) class during model training. The experimental results demonstrate that the performance of this model over other state-of-the-art models. Chen et al. (Chen et al., 2021) proposed a class-imbalanced deep learning method for image classification. In this work, authors used ensemble learning into CNN, and used a loss function to correct the preferences toward the majority classes. Fernando et al. (Fernando and Tsokos, 2021) presented a dynamic weighted balance loss function for imbalanced data learning. Using the dynamic weighting scheme, the model can be adjusted for different difficulty level of instances to achieve gradient update driven by hard minority class samples.

To summarize, significant advancements have been made in the field of imbalanced data learning in recent years. However, when it comes to machine learning models, training them with class-imbalanced time series data poses a formidable challenge. Initially, shallow machine learning models were predominantly employed for this task. In 2014, Cao et al. (Cao et al., 2014) proposed a method to address the issue of imbalanced time-series classification. In this work, authors used a statistical model, known as mixture of Gaussian trees, to model the possibly multimodal minority class. Priyadarsini et al. (Priyadarsini et al., 2021) presented a multi-class time series classification algorithm for imbalanced dataset. In recent years, some deep-learning-based time series classification methods have started to address the issue of imbalanced data distribution. Cheng et al. (Cheng et al., 2021) proposed a deep class-imbalanced semi-supervised model for detecting wind turbine blade icing. In this study, the authors combined semi-supervised learning and class-imbalanced learning techniques to construct a network model that can effectively balance classes for both labeled and unlabeled data samples. Wang et al. (Wang et al., 2017a) introduced a method that utilizes CNNs to extract features from vital sign time series, followed by extending feature vectors using categorical feature embedding. These augmented features are then fed into a multi-layer perceptron (MLP) for early disease prediction.

3. The cost-sensitive hybrid network model

In this section, we introduce the proposed cost-sensitive hybrid network model in detail. In general, deep network has strong feature extraction ability. However, for class-imbalanced time series datasets, there are two important issues that must be addressed. 1) the single CNN network cannot learn the sequential features of time series, while the long-term dependence of the time series should be considered; 2) the data imbalance problem must be addressed. The negative impact caused by skewed data distribution is that it is difficult to learn the effective feature representations due to the under-represented samples of the minority classes, so that the model tends to favor the majority class in actual detection (which may result in higher detection accuracy for the majority class and worse detection accuracy for the minority class). To solve these problems, we adopt a two-flow structure and introduce a cost-sensitive loss function to establish a novel hybrid network model, named cost-sensitive hybrid network (CSHN) model. The proposed CSHN model consist of a temporal flow and a spatial flow. The temporal flow is a CNN-based backbone network that is used to learn the local state features of time series from spatial perspective. The temporal flow uses the GRU network (Chung et al., 2014) to learn the sequential features of time series and force the decision logic of the network to pay

more attention to the long-term dependence of the time series. Then these features are fused and sent to a SoftMax for Anomaly Detection. During model training, we introduce a cost-sensitive loss function to measure the similarity between the outputted results and the ground-truth. Unlike general cross-entropy loss-based learning algorithms that minimize the classification error by minimizing the classification loss, cost-sensitive learning aims at minimizing the classification cost by giving different penalties (weights) to different classification errors so that the total classification cost is minimized. Moreover, it does not increase the computational complexity while regulating the impact of unbalanced data distribution. The overall architecture of the proposed network model is shown in Fig. 1.

3.1. The convolutional neural network

In the proposed CSHN model, we construct CNN using the convolutional layer and normalized layer, and use the Global Average Pooling (GAP) (Lin et al., 2013) in front of the output layer to reduce the dimension of the output value. By this way, the network parameters can be optimized, and thus the efficiency of the network model can be improved. Therefore, in our CSHN model, the CNN contains three important operations, convolution, batch normalization, and global average pooling. The specific calculation process is described as follows.

3.1.1. Convolution operation

The purpose of the convolution operation is to learn the local features. Let $W^l_{p_{l-1},p_l}$ represents the convolution kernel between p^{th}_l channel in l^{th} layer and p^{th}_{l-1} channel in $(l-1)^{th}$ layer, $\widehat{S}^{l-1}_{p_{l-1}}$ represents the outputted feature map of the p^{th}_{l-1} channel of sample S in $(l-1)^{th}$ layer, the convolution operation is working as follows.

$$\widehat{S}^l_{p_l} = \sum_{p_{l-1}=1}^{P} conv\left(\widehat{S}^{l-1}_{p_{l-1}}, W^l_{p_{l-1},p_l}\right) + b^l_{p_l} \tag{1}$$

Here, $\widehat{S}^l_{p_l}$ represents the outputted feature map of the $S=(t_1,\ldots,t_e)$ channel in $S=(t_1,\ldots,t_e)$ layer, $b^l_{p_l}$ represents a bias of the $S=(t_1,\ldots,t_e)$ channel in $S=(t_1,\ldots,t_e)$ layer, $S=(t_1,\ldots,t_e)$ represents convolution operation. P represents the number of convolution kernels in previous layer.

3.1.2. Batch normalization

In the processing of network training, batch normalization (BN) (Ioffe and Szegedy, 2015) refers to adjust the intermediate output of the neural network using the mean and standard deviation of small batches, so that the value of intermediate output in each layer tends to be stable. Thus, the problem of the unstable data distribution can be solved. Therefore, it is necessary to normalize the data before inputting into the network. Batch normalization can improve the stability and generalization ability of the network.

For time series sample $S = (t_1, \ldots, t_e)$, the BN operation can be expressed as follows.

$$\mu = \frac{1}{e}\sum_{m=1}^{e} t_m, m = \{1,2,\cdots,e\} \tag{2}$$

$$\sigma^2 = \frac{1}{e}\sum_{m=1}^{e}(t_m-\mu)^2 \tag{3}$$

$$\widehat{t}_m = \frac{t_m-\mu}{\sqrt{\sigma^2+\varepsilon}} \tag{4}$$

Here, μ and σ represent the mean and variance, respectively, $\widehat{t}_m$ represents the standard normalized value. $\varepsilon > 0$ is a small constant, and it guarantees that the denominator is greater than 0.

Considering the distribution trend that is outputted from the activation function may not be standard normal distribution with mean 0 and variance 1, we introduce parameters $\gamma^{(m)}$ and $\rho^{(m)}$ to accurately represent the distribution trend of data that is outputted in each layer of the network.

$$y^{(m)} = \gamma^{(m)}\widehat{t}_m + \rho^{(m)} \equiv BN_{\gamma,\rho}(t_m) \tag{5}$$

Here, $\gamma^{(m)}$ represents the scale of data change, $\rho^{(m)}$ represents the offset. $y^{(m)}$ represents the normalized value of t_m.

3.1.3. Global average pooling

In general, to reduce the dimension of feature vector obtained from convolution operation of the last layer, the convolutional network usually includes one or more fully connected layers near the outputted layer, shown in Fig. 2. In this study, as shown in Fig. 3, we use a GAP layer to replace the fully connected layer. By this way, both the dimension of feature vector and the numbers of network parameters can be reduced.

Using GAP layer, the average pooling operation on the multiple feature vectors obtained from last convolution layer can be achieved as follows.

$$g_q = conv(\widehat{S}_q, W_{GAP}), q = \{1,\cdots,Q\} \tag{6}$$

$$Y = \{g_1, g_2, \cdots, g_Q\} \tag{7}$$

Here, $\widehat{S}_q$ represents the feature map of q^{th} channel of the sample S, it is the output result of previous convolution layer, W_{GAP} represents the average pooling matrix, Q represents the number of channels in the last

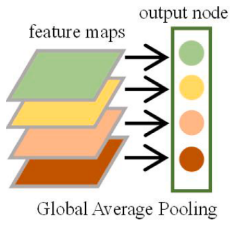


Fig. 2. Full connection layer.

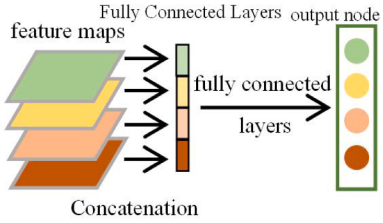


Fig. 3. Global average pooling.

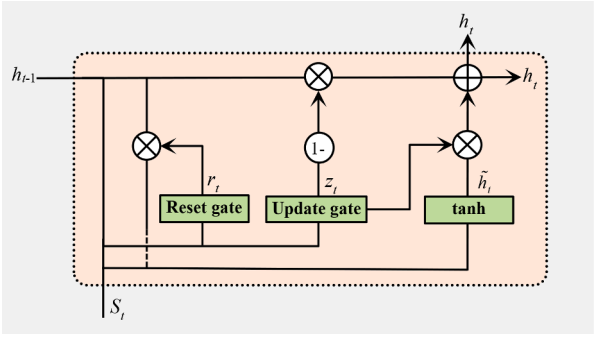


Fig. 4. The Schematic diagram of GRU Network structure.

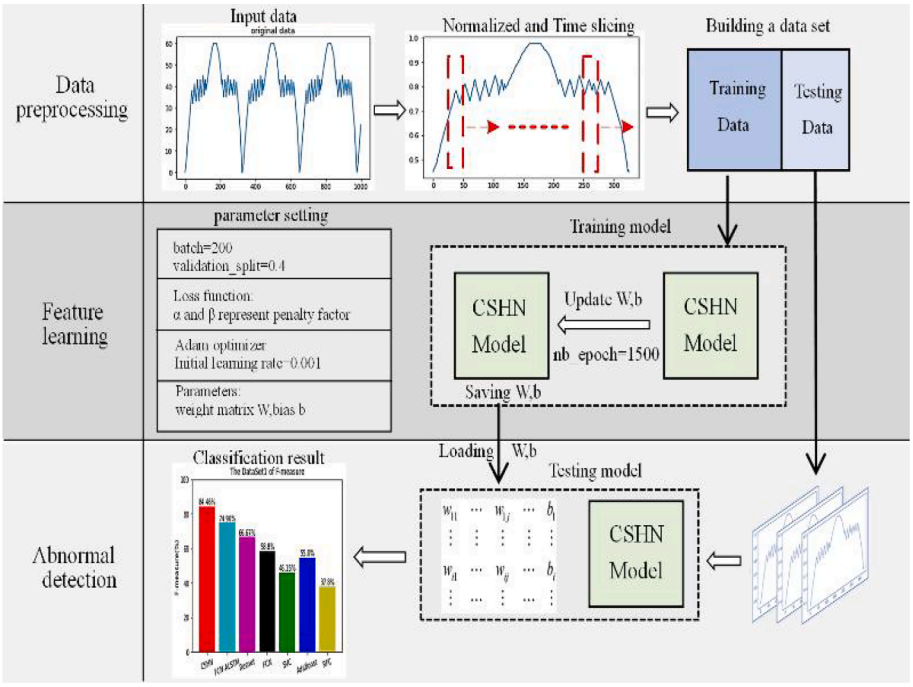


Fig. 5. The Schematic diagram of the proposed method.

convolution layer. Y denotes the resulting feature map.

3.1.4. Gated recurrent unit network

The GRU is a variant of the Long Short-Term Memory (LSTM). The latter is a complex network structure consisting of a forgotten gate, an input gate, and an output gate. GRU network is the simplified form of the LSTM recurrent neural network. It is composed of an update gate z_t and a reset gate r_t, where z_t is obtained by combining the forgetting gate and the input gate in the LSTM network. The structure of the GRU network is shown in Fig. 4.

In Fig. 4, S represents the time series data. h_t represents the output information at time t, $\widetilde{h}_t$ represents the hidden state at time t. The process of feature learning from GRU network is affected by the outputted value h_{t-1} of the previous moment and the inputted value S_t of current time. Therefore, at time t, the inputs are h_{t-1} and S_t.

The reset gate r_t controls the amount of information that the outputted value h_{t-1} flow into hidden state $\widetilde{h}_t$. r_t is mapped to [0, 1] by Sigmoid function. The smaller the value r_t is, the information that flows into $\widetilde{h}_t$ is less. The hidden state $\widetilde{h}_t$ is mapped to [-1, 1] by tanh activation function. This process can be expressed as follows.

$$r_t = \sigma(W_r \cdot [h_{t-1}, S_t]) \tag{8}$$

$$\widetilde{h}_t = \tanh(W_{\tilde{h}} \cdot [r_t \cdot h_{t-1}, S_t]) \tag{9}$$

Here, W_r is the weight matrix of the reset gate, $[h_{t-1}, S_t]$ represents the operation that connects two inputted vectors h_{t-1} and S_t into a long vector, σ represents the activation function, and $W_{\tilde{h}}$ represents the weight for calculating the hidden state.

The update gate z_t determines the probability of incorporating the outputted information h_{t-1} into the final output. h_t. Here, $z_t \in [0,1]$. The larger the value z_t is, the information that is carried into h_t from h_{t-1} is less. This process can be mathematically represented as follows.

$$z_t = \sigma(W_z \cdot [h_{t-1}, S_t]) \tag{10}$$

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \widetilde{h}_t \tag{11}$$

Here, W_z represents the weight matrix of the update gate.

3.1.5. Design of loss function

In general, during model training, a cross-entropy loss function is used to measure the similarity between predicted value and the ground-truth. Such a cross-entropy loss function can be represented as follows.

$$\widetilde{L}(W, b) = -\left(\sum_{k=1}^{n} [y_k \cdot \log(h_{W,b}(S_k)) + (1 - y_k) \cdot \log(1 - h_{W,b}(S_k))]\right) \tag{12}$$

Here, y_k represents the label of the k^{th} real sample, S_k represents inputted k^{th} time series sample, $h_{W,b}(S_k)$ represents the probability value, W represents the weight parameter, b represents the bias, and n represents the total number of samples. The role of the cross-entropy loss function is adjusting parameters W and b to minimize the total loss value $\widetilde{L}(W, b)$. In general, the smaller the total loss $\widetilde{L}(W, b)$ is, the better the learning effect of the model will be. In the case of severely skewed data distribution, the network model is difficult to learn appropriate features from minority class samples by using general cross entropy loss function. The reason is that the classical cross entropy loss provides with equal importance for each data instance. Assume that all samples are predicted to be majority classes samples, the total loss will not increase because the sample number of minority class is too little. As a result, the classification accuracy for minority class will be inaccurate. Therefore, the general cross entropy loss function is not suitable for classification tasks of class-imbalance data.

To solve above problem, we construct a new "cost sensitive" loss function that uses different penalty factors to penalize the misclassification for network model. When the minority class samples are misclassified, we use larger penalty factor α to enlarge the total loss. When majority class samples are misclassified, we use smaller penalty factor β to control the increasing of the total loss. Owing to the large number of the majority class samples, the total loss will be large even if using a smaller penalty factor. The proposed cost-sensitive loss function can be represented as follows.

$$\widetilde{L}(W, b) = -\left(\sum_{k=1}^{n} [\alpha \cdot y_k \cdot \log(h_{W,b}(S_k)) + \beta \cdot (1 - y_k) \cdot \log(1 - h_{W,b}(S_k))]\right) \tag{13}$$

Here, α represents the penalty factor for minority class samples to be misclassified. β represents the penalty factor for majority class samples

to be misclassified.

4. Time series anomaly detection algorithm based on the proposed CSHN model

In this section, we present an anomaly detection method for time-series via using the proposed CSHN model. Our method consists of three phases: data preprocessing, feature learning and anomaly detection. Data preprocessing mainly includes data normalization and time slicing. In feature learning stage, we use CSHN model to learn the feature of the time-series data. Then we use the trained CSHN model to perform anomaly detection. The schematic diagram of our method is shown in the Fig. 5.

4.1. Data preprocessing

4.1.1. Data normalization

In practical engineering, the magnitude of measured data may be inconsistent. In model training, the data with larger magnitude is dominant absolutely, and is the main factor that leads to slow convergence speed for network model. Therefore, it is necessary to normalize the raw data to obtain a dataset with consistent scales.

Let $T\{t_i(x_i,y_i)\}(i=1,2,\cdots,N)$ represents the sample datasets. Here, x_i represents the signal value of *i-th* moment, y_i represents the label of x_i (y_i is 0 or 1), and N represents the length of the sequence. Data normalization can be expressed as follows.

$$\overline{x}_i = \frac{(x_i - \overline{X})}{var(X)} \tag{14}$$

Here, $\overline{X} = \frac{1}{N}\sum_{i=1}^{N} x_i$, $var(X) = \frac{1}{N}\sum_{i=1}^{N}(x_i-\overline{X})^2$, Let $\overline{T}\{t_i(\overline{x}_i,y_i)\}$ represents normalized sequence.

4.1.2. Time slicing

To facilitate network training, we adopt a sliding window to slice the long time series into short overlapping fragments. Taking a window function $Slicing(\cdot)$ with length L, and L is set as one half of the data period. The moving step is S, the normalized data $\overline{T}\{t_i(\overline{x}_i,y_i)\}$ is sliced into a data set $\{S_1,S_2,\cdots,S_{\overline{N}}\}$, here, the length of each component $S_a(a=1,\cdots\overline{N})$ is L.

$$Slicing\left(\overline{T}\{t_i(\overline{x}_i,y_i)\}\right) = \begin{bmatrix} S_1 \\ S_2 \\ \vdots \\ S_a \\ \vdots \\ S_{\overline{N}} \end{bmatrix} = \begin{Bmatrix} [t_1(x_1,y_1) : t_L(x_L,y_L)] \\ [t_{s+1}(x_{s+1},y_{s+1}) : t_{L+s+1}(x_{L+s+1},y_{L+s+1})] \\ \vdots \\ \left[t_{(a-1)\cdot s+1}\left(x_{(a-1)\cdot s+1},y_{(a-1)\cdot s+1}\right) : t_{(a-1)\cdot s+1+L}\left(x_{(a-1)\cdot s+1+L},y_{(a-1)\cdot s+1+L}\right)\right] \\ \vdots \\ [t_{N-L+1}(x_{N-L+1},y_{N-L+1}) : t_N(x_N,y_N)] \end{Bmatrix} \tag{15}$$

4.2. Feature learning

We take 80 % data $S_{train\_set} \subseteq \{S_1,S_2,\cdots,S_{\overline{N}}\}$ as the training samples, and input into the CSHN. We use Adam (Kingma and Ba, 2014) as the optimizer. The learning rate is set to 0.001, the gradient is decreased every 200 fragments, and the cross-validation is performed using 40 % of the training samples. In training, we use the back-propagation algorithm (Lecun et al., 2012) to update model parameters, and set the number of iterations to 1500 times. The specific process of feature learning is described as follows.

4.2.1. Feature learning of the convolution layer

In the proposed CSHN model, the hidden layer of CNN consists of three convolutional layers, and each convolutional layer contains three processing operations. The specific process is as follows.

Layer $C1$: Assume that layer $C1$ contains n_1 convolution kernels $W^1_{p_1}(p_1=1,2,\cdots,n_1)$, and the size of each convolution kernel is c_1. Let $n_1=32,\ c_1=8$. For the input $S_a(S_a\in S_{train\_set})$, after convolutional operation, 32 feature vectors $\widehat{S}^1_{a,p_1}(p_1=1,2,\cdots,32)$ can be obtained, and the length of each feature vector is $L-7$. Finally, using *BN* operation and the activation function $LeakyReLU(\cdot)$, the final output $\widehat{y}^1_{a,p_1}$ can be obtained. This process can be expressed as follows.

$$\widehat{S}^1_{a,p_1} = conv(S_a, W^1_{p_1}) + b^1_{p_1} \tag{16}$$

$$y^1_{a,p_1} = BN(\widehat{S}^1_{a,p_1}) \tag{17}$$

$$\widehat{y}^1_{a,p_1} = LeakyReLU\left(y^1_{a,p_1}\right) \tag{18}$$

Here,$b^1_{p_1}$ represents the bias, $conv(\cdot)$ represents the convolution operation.

Layer $C2$: Assume that layer $C2$ has n_2 convolution kernels $W^2_{p_1,p_2}(p_1=1,\cdots,32,\ p_2=1,\cdots,n_2)$, and the size of each convolution kernel is c_2. Let $n_2=64, c_2=5$. After convolutional operation, 64 feature vectors $\widehat{S}^2_{a,p_2}(p_2=1,2,\cdots,64)$ can be obtained, and the length of each feature vector is $L-11$. Finally, using *BN* operation and the activation function $LeakyReLU(\cdot)$, the final output $\widehat{y}^2_{a,p_2}$ can be obtained. This process can be expressed as follows.

$$\widehat{S}^2_{a,p_2} = \sum_{p_1=1}^{32} conv(\widehat{y}^1_{a,p_1}, W^2_{p_1,p_2}) + b^2_{p_2} \tag{19}$$

$$y^2_{a,p_2} = BN(\widehat{S}^2_{a,p_2}) \tag{20}$$

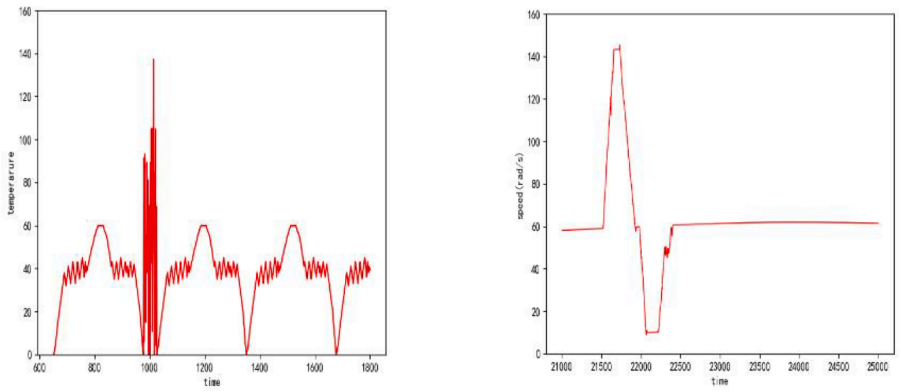
$$\widehat{y}^2_{a,p_2} = LeakyReLU\left(y^2_{a,p_2}\right) \tag{21}$$

Here, $b^2_{p_2}$ represents the bias of the layer $C2$.

Layer $C3$: Assume that layer $C3$ has n_3 convolution kernels $W^3_{p_2,p_3}(p_2=1,\cdots,64,\ p_3=1,\cdots,n_3)$, and the size of each convolution kernel is c_3. Let $n_3=128, c_3=3$. After convolutional operation, 128 feature vectors $\widehat{S}^3_{a,p_3}(p_3=1,2,\cdots,128)$ can be obtained, and the length of each feature vector is $L-13$. The final output $\widehat{y}^3_{a,p_3}$ of layer $C3$ can be generated using *BN* operation and the activation function $LeakyReLU(\cdot)$. This process can be expressed as follows.

$$\widehat{S}^3_{a,p_3} = \sum_{p_2=1}^{64} conv(\widehat{y}^2_{a,p_2}, W^3_{p_2,p_3}) + b^3_{p_3} \tag{22}$$

$$y^3_{a,p_3} = BN(\widehat{S}^3_{a,p_3}) \tag{23}$$



(a) The data change trend of DataSet1 (b) The data change trend of DataSet2

Fig. 6. The example of data change trend.

$$\widehat{y}^3_{a,p_3} = LeakyReLU\left(y^3_{a,p_3}\right) \tag{24}$$

Here, $b^3_{p_3}$ represents the bias of the $C3$ layer.

Layer GAP: For the feature vector $\widehat{y}^3_{a,p_3}$, the GAP is performed using the convolution kernel W_{GAP} to generate a 128-dimensional feature vector Y^{CNN}_a.

$$g_{p_3} = conv(\widehat{y}^3_{a,p_3}, W_{GAP}), p_3 = \{1, \cdots, 128\} \tag{25}$$

$$Y^{CNN}_a = \{g_1, g_2, \cdots, g_{128}\} \tag{26}$$

Here, g_{p_3} represents the component of feature vector Y^{CNN}_a.

4.2.2. Feature learning of the GRU network

For the inputted time series sample $S_a(S_a \in S_{train\_set})$, we use the GRU network with 128 cells to learn the sequence feature, and output feature vector Y^{GRU}_a. It can be expressed as follows.

$$Y^{GRU}_a = f_{GRU}(S_a; W_r, W_z) \tag{27}$$

Here, W_r represents the weight matrix of the reset gate, W_z represents the weight matrix of the update gate, $f_{GRU}(\cdot)$ represents the mapping function of the GRU network.

4.2.3. Output of the CSHN model

For the inputted time series sample $S_a(S_a \in S_{train\_set})$, the CSHN model will eventually output a probability value $Pr_{n_{class}}(S_a)$ via using SoftMax. Here $n_{classes}$ is 0 or 1. Let $n_{class} = 0$ denotes that S_a belongs to the majority class, and $n_{class} = 1$ denotes that S_a belongs to the minority class. This process can be expressed as follows.

$$Pr_{n_{class}}(S_a) = Softmax\left(concatenate\left[Y^{CNN}_a, Y^{GRU}_a\right]\right) \tag{28}$$

Here, Y^{CNN}_a represents outputted feature vector from CNN, Y^{GRU}_a represents outputted feature vector by GRU. The function $concatenate(\cdot)$ concatenates Y^{CNN}_a and Y^{GRU}_a into a long vector.

4.2.4. Cost-sensitive loss function

We use the proposed cost sensitive loss function (formula (13)) to adjust the outputting of the CSHM model, and use Adam optimization algorithm as back-propagation algorithm to minimize the training loss, to obtain optimal weight parameters. In this sense, formal (13) is transformed into the following form:

$$\widetilde{L}(W, \mathrm{b}) = -\left(\sum_{a=1}^{\overline{N}}[\alpha \cdot y_a \cdot \log(Pr_1(S_a)) + \beta \cdot (1 - y_a) \cdot \log(Pr_0(S_a))]\right) \tag{29}$$

Here, $W = \{W^1_{p_1}, W^2_{p_1,p_2}, W^3_{p_2,p_3}, W_{GAP}, W_r, W_z\}$, $b = \{b^1_{p_1}, b^2_{p_2}, b^3_{p_3}\}$. Using formula (29), the weights W and the bias b can be updated through back-propagation mechanism of the Adam optimization algorithm. Here, the learning rate is 0.001, the gradient is decreased every 200 fragments, and the cross-validation is performed by using 40 % of the training samples.

The final weights W and the bias b are closely related to the penalty factor α and β. When the minority class samples are misclassified, we enlarge the total loss by using larger penalty factor α. When majority class samples are misclassified, we control the increasing of total loss by using a smaller penalty factor β.

The definition of the penalty factors α and β is as follows.

$$\alpha = \frac{N}{n_{classes} \cdot n_{abnormal\_total}}, \ \beta = \frac{N}{n_{classes} \cdot n_{normal\_total}} \tag{30}$$

Where N is the total number of samples, n_{normal_total} is the number of normal samples, $n_{abnormal\_total}$ is the number of abnormal samples, and $n_{classes}$ is the number of category, in this study, $n_{classes} = 2$.

The proposed cost sensitive loss function can be generalized to the case of multiple classifications, in which, the penalty factors for multiple types of imbalanced data samples can be set in Eq. (31).

$$\alpha_1 = \frac{N}{n_{classes} \cdot n_{1\_total}}, \ \alpha_2 = \frac{N}{n_{classes} \cdot n_{2\_total}}, \ \cdots, \ \alpha_{classes} = \frac{N}{n_{classes} \cdot n_{i\_total}} \tag{31}$$

Here, n_{i_total} is the number of samples in i^{th} class, α_i correspond to the punishment factors of the i^{th} class, $i = \{1, 2, \cdots, n_{classes}\}$.

4.2.5. Abnormal detection

In anomaly detection phase, testing data is feed to the trained CSHN model. We take 20 % data $S_{test\_set} \subseteq \{S_1, S_2, \cdots, S_{\overline{N}}\}$ as the testing samples. Let $H(S_a; W, b)$ represents the CSHN model. Here, $S_a \in S_{test\_set}$.

$$H(S_a; W, b) => Pr_{n_{class}}(S_a) \tag{32}$$

$$y_{a\_label} = \begin{cases} 0, & if \quad Pr_0(S_a) \geqslant Pr_1(S_a) \\ 1, & if \quad Pr_0(S_a) < Pr_1(S_a) \end{cases} \tag{33}$$

Here, $Pr_{n_{class}}(S_a)$ is predicted probability value, y_{a_label} is the label of the predicted sample, and $W = \{W^1_{p_1}, W^2_{p_1,p_2}, W^3_{p_2,p_3}, W_{GAP}, W_r, W_z\}$, $b = \{b^1_{p_1}, b^2_{p_2}, b^3_{p_3}\}$ are the parameters obtained from the learning process.

5. Experimental results and analysis

We discuss the performance of the proposed method via experiments. Our method is implemented and tested using TensorFlow 1.10.0, anacoda3-5–1.0 and keras2.22. We run programs in the computer with i7-6800 K CPU, GTX1080Ti GPU, 32.00 GB RAM. Our experiments are performed on two types of datasets: the real-world engineering datasets, and 44 UCR benchmark datasets. We assume that majority class represents normal class, minority class represents abnormal class. In the

Table 1
Detection Results on the DataSet1.

Network model	ACC^+	ACC^-	G-means	F-measure
CNN + GRU + Cross entropy loss function	0.9790	0.7894	0.8791	0.7894
CNN + GRU + Cost sensitive loss function	**0.9895**	**0.8314**	**0.9070**	**0.8446**

Table 2
Detection results on the DataSet2.

Network model	ACC	ACC	G-means	F-measure
CNN + GRU + Cross entropy loss function	**0.9956**	0.7865	0.8848	0.7915
CNN + GRU + Cost sensitive loss function	0.9951	**0.8876**	**0.9398**	**0.8826**

Table 3
Parameter setting for the DataSet1.

model	parameter settings
SVC	The coefficient of the penalty term C is set as 3, and the kernel function is set as the radial basis function (rbf), gamma = 2.
AdaBoost	The number of basic classifiers is set as 110.
RFC	The number of decision trees is set as 130, and the maximum tree depth is set as 8.

Table 4
Parameter setting for the DataSet2.

model	parameter settings
SVC	The coefficient of the penalty term C is set as 2, and the kernel function is set as the radial basis function (rbf), gamma = 1.8.
AdaBoost	The number of basic classifiers is set as 180.
RFC	The number of decision trees is set as 150, and the maximum tree depth is set as 5.

experiments, the FCN_ALSTM network (Karim et al., 2017), Resnet network (Wang et al., 2017), FCN network (Wang et al., 2017), and the proposed CSHN are implemented by using Keras deep learning package (Https://keras.io/, n.d). The Support Vector Classification (SVC), AdaBoost, Random Forest Classifier (RFC) algorithms are performed using the Scikit-learn machine learning package (Http://scikit-learn.org/, n.d) in Python 3.5. To avoid the impact of randomness, each experiment is run 10 times and then the statistical average values are presented.

5.1. Dataset

In this study, the experiments are conducted on two types of datasets: the real-world engineering datasets DataSet1 and DataSet2, and 44 UCR benchmark datasets.

DataSet1 and DataSet2 comes from practical engineering measurement, consist of raw engineering time series data. The DataSet1 contains 445,403 pieces speed measurements obtained from a flywheel installed on a specific device, and the DataSet2 contains 83,996 temperature measurements obtained from a gyroscope installed on the same device. In these datasets, the majority of data typically falls within the normal range, with only a minimal amount of data falling outside the normal range, which can be considered as abnormal values. Furthermore, abnormal data demonstrates a state of mutation and deviates from established statistical principles. Fig. 6 illustrates a segment depicting the trend in data changes, where the abscissa represents the time, and the ordinate represents the corresponding signal value.

We used sliding window (the window step length is one half of the data period) to divide two long time series into short overlapping fragments. The number of fragments in dataset1 is 5936, and the number of fragments in dataset2 is 1118. For dataset1, we use 4748 fragments as training set, and 1188 fragments as test set; for dataset2, we use 894 fragments as training set, and 224 fragments as test set.

5.1.1. Evaluation metric

We use true positive (TP), false negative (FN), true negative (TN), and false positive (FP) to evaluate the performance of the proposed method. The definitions of these indicators are as follows.

TP: The normal value is detected as normal value.

FN: The normal value is detected as abnormal value.

FP: The abnormal value is detected as normal value.

TN: The abnormal value is detected as abnormal value.

We evaluate the performance of our method using following indicators: Recall and Precision, *G*-means and *F*-Measure. ACC^+ and ACC^- represent the detection rate of normal samples and abnormal samples, respectively, that is, ACC^+ represent the true positive rate (*TPR*), ACC^- represent the true negative rate (*TNR*). *G*-means can comprehensively evaluate the detection performance of the method. These indicators are defined as follows.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{34}$$

$$ACC = TPR = Recall = \frac{TP}{TP + FN} \tag{35}$$

$$G - means = \sqrt{ACC^+ \times ACC^-} \tag{36}$$

F-measure: *F*-measure is an indicator to measure the classification performance of the classifier for abnormal class. It is also a comprehensive evaluation indicator. It is defined using the weighted harmonic average of Recall and Precision.

$$F\text{ - }measure = \frac{(1 + \beta^2) \times \text{Recall} \times \text{Precision}}{\beta^2 \cdot \text{Recall} \ + \ \text{Precision}} \tag{37}$$

Here, β is a coefficient that adjusts the relative importance of precision versus Recall (if $\beta = 1$, *F*-measure will be F_1-score).

5.2. Detection accuracy

To investigate the effectiveness of CSHN model, we compare the influence of the cost sensitive loss function with general cross-entropy loss function on the detection accuracy. To this end, we conduct experiments by using the proposed Hybrid network model (CNN + GRU) combine with the cost sensitive loss function and general cross-entropy loss function, respectively. The experiments are performed on the DataSet1 and DataSet2, respectively. The results are summarized in the Tables 1 and 2.

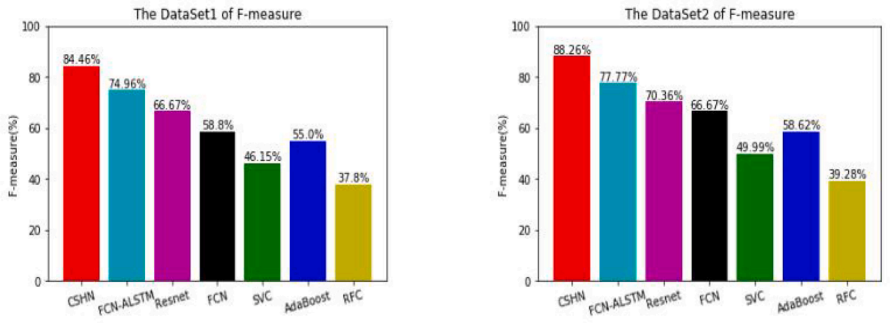
We can see from the Tables 1 and 2, for the normal samples, the changing range of ACC^+ is very small. For abnormal samples, the values of ACC^- are only about 78 % in the case of using the general cross-entropy loss function, while, in the case of using the proposed cost-sensitive loss function, the values of ACC^- are increased nearly 5 % – 10 %. It means that the detection accuracy of the minority class is significantly improved, which means that the proposed cost-sensitive loss function can be used to solve the problem of the low classification accuracy for minority classes caused by skew data distribution. Moreover, from the G-means and F-measure, we can see that the detection performance has been improved by using the cost-sensitive loss function. The reason is the using of different penalty factors to penalty the misclassification of the network model. When the minority class samples are misclassified, we use larger penalty factor to enlarge the total loss. When majority class samples are misclassified, we use smaller penalty factor to control the increasing of total loss.

Table 5
ACC^{+}, ACC^{-} and G-means of different methods on the DataSet1.

Precision	CSHN	FCN_ALSTM	Resnet	FCN	SVC	AdaBoost	RFC	Knn+DTW
ACC	0.9895	**0.9947**	0.9842	0.9738	0.9424	0.9476	0.9476	0.9798
ACC	**0.8314**	0.631	0.5789	0.526	0.4737	0.5789	0.3684	0.7058
G-means	**0.9070**	0.7922	0.7548	0.7157	0.6681	0.7407	0.5908	0.8664

Table 6
ACC^{+},ACC^{-} and G-means of different methods on the DataSet2.

Precision	CSHN	FCN_ALSTM	Resnet	FCN	SVC	AdaBoost	RFC	Knn+DTW
ACC	**0.9951**	0.9855	0.9782	0.9746	0.9420	0.9565	0.9420	0.9877
ACC	**0.8876**	0.7241	0.6551	0.6206	0.5172	0.5862	0.3793	0.8740
G-means	0.9398	0.8447	0.8005	0.7777	0.6979	0.7488	0.5977	**0.9407**



(a)Detection results of F-measure on DataSet1

(b) Detection results of F-measure on DataSet2

Fig. 7. The comparison of F-measure.

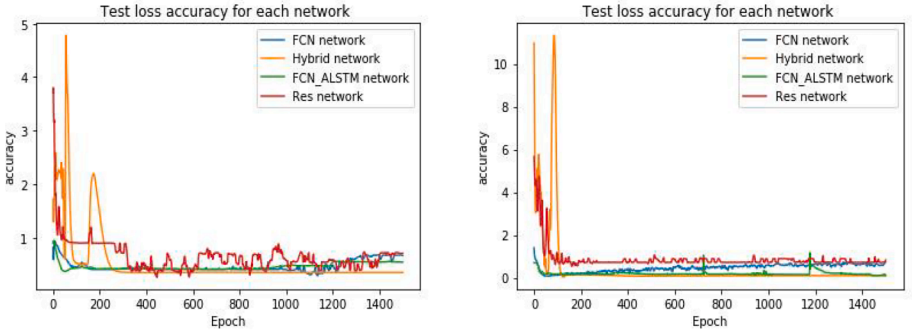
5.3. Performance comparison

To fully investigate the time series data anomaly detection methods, we compare the CSHN model with the deep-learning based models FCN_ALSTM network, Resnet network, FCN network, and traditional machine-learning based methods SVC, AdaBoost and RFC, and the typical method such as K Nearest Neighbors & Dynamic Time Warping (KNN-DTW). The experiments are conducted on the DataSet1 and DataSet2. In the experiment, we use the methods of cross-validation and grid search to select the hyperparameters. Here, we use 40 % training samples as the cross-validation data set, the step size of the gradient decreased is 200 samples (batch = 200), and the learning rate is 0.001. To facilitate the comparison of deep network models, the maximum number of iterations is set as 1500. For the SVC model, AdaBoost model and RFC model, the experimental hyperparameters are set in the Tables 3 and 4.

5.3.1. Evaluation and comparison for ACC+, ACC- and G-means

To evaluate the detection accuracy, we investigate the evaluation metrics ACC^{+},ACC^{-} and G-means on DataSet1 and DataSet2, the results are listed in Tables 5 and 6.

As can be seen from the Tables 5 and 6, the deep-learning-based methods are superior to the machine-learning-based algorithms. For the normal samples, the detection results of the ACC^{+} are more than 94 % for all tested method. In these methods, FCN_ALSTM network has higher detection accuracy compared with other methods. The reason is that the FCN_ALSTM network model contains the attention mechanism, which can learn the prominent features of all samples. While, for abnormal samples, the ACC^{-} values of the proposed method are greater than that of the comparing methods. Comparing with the best method FCN_ALSTM, the ACC^{-} value of the proposed method is improved more than 20 %. Though the G-mean of the proposed method is slightly lower



(a) On DataSet1

(b) On DataSet2

Fig. 8. The change curve of the training loss values. (Hybrid network is the proposed CSHN model).

Table 7
Accuracy and MPCE of different methods on 44 UCR datasets.

Dataset	Nbclasses	CSHN	ALSTM	FCN	Resnet	Adboost	SVC	RFC
Adiac	37	0.8391	**0.8432**	0.7698	0.8209	0.7519	0.7519	0.6035
Beef	5	**0.9666**	0.8666	0.73	0.7666	0.9	0.9	0.7333
ChlorineCon	3	0.9125	0.8736	0.8226	0.8315	**0.9157**	0.9057	0.6429
CBF	3	0.9223	0.94	**1.0**	0.9977	0.8733	0.8833	0.8622
CinCECGtorso	4	0.7753	0.7434	0.6434	0.805	**0.8432**	0.8123	0.7057
Coffee	2	**1.0**	**1.0**	**1.0**	**1.0**	0.9285	**1.0**	0.9642
Cricket_X	12	0.7948	0.4538	0.7461	**0.7974**	0.6948	0.5974	0.5538
Cricket_Y	12	0.8128	0.4564	0.7666	**0.8153**	0.5846	0.5974	0.5923
Cricket_Z	12	**0.8358**	0.482	0.7974	0.7948	0.5256	0.5974	0.5743
DiatomSizeR	4	0.9803	0.9411	0.9509	**0.9901**	0.9215	0.8856	0.879
ECGFiveDays	2	**0.9988**	0.9756	0.9883	0.9454	0.7305	0.9814	0.7711
FaceAll	14	0.8976	0.8804	**0.9195**	0.8272	0.8491	0.7893	0.8236
FaceFour	4	0.9204	0.8522	**0.9318**	0.9545	0.6931	0.8522	0.7727
FacesUCR	4	0.8341	0.8092	0.9458	**0.9609**	0.7492	0.8063	0.7663
50words	14	0.6923	0.8012	0.5604	0.7472	0.6321	0.5012	0.4023
FISH	50	0.92	0.8228	0.9657	**0.9942**	0.8171	0.8914	0.7714
Gun_Point	7	0.9533	**1.0**	**1.0**	0.9433	0.8666	0.9533	0.9266
Haptics	2	0.4805	**0.522**	0.4188	0.5	0.3474	0.435	0.4318
InlineSkate	5	0.3636	0.3145	0.2654	0.32	**0.3781**	0.309	0.3218
ItalyPower	7	**0.9727**	0.9562	0.9611	0.9591	0.9708	0.9591	0.9669
Lighting2	2	0.7049	0.7049	0.7049	0.754	0.7049	0.7049	**0.7704**
Lighting7	2	0.6849	0.726	0.7808	**0.8493**	0.5068	0.6301	0.6849
MALLAT	7	0.9455	**0.9862**	0.962	0.9637	0.8874	0.9501	0.8179
MedicalImages	10	0.7368	0.7289	**0.7618**	0.7473	0.6776	0.7368	0.6671
MoteStrain	2	0.8961	0.869	**0.9353**	0.8889	0.8099	0.845	0.8706
NonInvThorax1	42	0.9477	**0.9778**	0.9456	0.9511	0.8954	0.8421	0.8101
NonInvThorax2	42	0.9399	**0.966**	0.9328	0.944	0.9012	0.8844	0.7923
OliveOil	4	0.8333	**0.9433**	0.8	0.8666	0.8333	0.8666	0.9333
OSULeaf	6	0.9115	0.5454	0.9504	**0.971**	0.5347	0.5826	0.5041
SonyAIBORobot	2	0.8768	**0.9635**	0.955	0.9484	0.7903	0.7487	0.6578
SonyAIBORobotII	2	0.8373	**0.9732**	0.9695	0.9621	0.8701	0.8153	0.7948
StarLightCurves	3	0.9592	**0.9786**	0.9063	0.9763	0.8932	0.9154	0.9425
SwedishLeaf	15	0.9508	**0.9676**	0.9584	0.9328	0.8556	0.8992	0.8432
Symbols	6	0.9764	**0.9839**	0.9356	0.9718	0.8321	0.8874	0.8201
syntheticControl	6	0.95	**0.9973**	0.9933	0.9966	0.9866	0.9633	0.9433
Trace	4	**1.0**	**1.0**	**1.0**	**1.0**	0.83	0.84	0.84
TwoLeadECG	2	0.9367	0.9043	**1.0**	0.9982	0.7304	0.8252	0.726
Two_Patterns	4	0.8902	0.891	0.886	**1.0**	0.835	**0.925**	0.81
uWaveX	8	0.8086	**0.8174**	0.7063	0.7867	0.7098	0.7036	0.7412
uWaveY	8	0.6876	**0.765**	0.553	0.6786	0.5495	0.701	0.6836
uWaveZ	8	0.704	**0.7937**	0.6945	0.7666	0.7545	0.725	0.7051
wafer	2	0.9962	0.9941	**0.9996**	0.997	0.9829	0.9954	0.9831
WordsSynonyms	25	**0.6597**	0.5689	0.4952	0.6473	0.5398	0.5799	0.547
yoga	2	**0.8733**	0.7953	0.6776	0.8583	0.724	0.756	0.799
Win		**8**	**18**	**10**	**10**	**3**	**2**	**1**
MPCE		**0.03464**	**0.036398**	**0.036364**	**0.02860**	**0.056673**	**0.04728**	**0.05659**
MPCE		**0.03464**	**0.036398**	**0.036364**	**0.02860**	**0.056673**	**0.04728**	**0.05659**

than Knn + DTW, the total detection performance of the proposed method is superior to the comparing methods.

5.3.2. *Investigation and comparison for F-measure*

Fig. 7(a and b) show the detection results of F-measure on DataSet1 and DataSet2, respectively.

As can be seen from the Fig. 7, the deep-learning-based methods are superior to the machine-learning-based algorithms. The reason is that the neural network has better feature representation ability for the nonlinear relation. We can see that the F-measure of our method is higher than that of the comparison methods. These results demonstrate that the detection performance of our method is superior to the comparing methods.

5.3.3. *Evaluation and comparison for the convergence speed and stability*

For the deep neural network, the training loss reflects the convergence speed and stability of the network model. We compare the CSHN model with the deep learning models FCN_ALSTM, Resnet, and FCN, in term of the loss accuracy. Fig. 8 shows the curves of the training loss on DataSet1 and DataSet2.

In Fig. 8(a and b) show the change tendency of the training loss on Dataset1 and Dataset2, respectively. The converging rate of the loss value is relatively fast. On the Dataset1, when the number of iterations more then 250, the loss value of the proposed CSHN model tends to stable, and obviously lower than that of the comparing network models. On the Dataset2, when the number of iteration times more then 120, the loss value of the proposed CSHN model is tend to stable, and lower than that of the comparing network models. This means that the stability of the proposed model is superior to the comparing network models.

5.4. *Performance evaluation on the UCR public datasets*

Although the proposed CSHN model is specifically designed for deal with the problem of class-unbalanced time series data classification, it is still effective for standard time series. To substantiate this assertion, we conducted experiments on UCI standard time series database. As an open and extensively utilized data resource in the field of machine learning, UCI undoubtedly stands as the most comprehensive and widely accepted benchmark for testing algorithms. The database contains many data sets, tasks, and evaluation criteria to help researchers and developers test, evaluate, and compare various machine learning algorithms. The UCI database encompasses datasets from diverse fields including statistics, biology, medicine, engineering, and social sciences. It adheres to a standard distribution where both majority and minority class samples

are relatively balanced. In this work, we utilize the UCI dataset to validate the effectiveness of the proposed CSHN model on standard datasets, despite it is specifically designed for addressing class-unbalanced time series classification problems. To this end, we conducted experiments on 44 UCR datasets (sufficient to support our claim).

Considering that the model is tested on multiple datasets, it is necessary to define new indicators to evaluate the comprehensive classification performance. To this end, we use the evaluation indicators Accuracy and Mean per-class Error (MPCE) to evaluate the performance of classification. Here, we define the data pool $D=\{d_j\}$, d_j represents the j^{th} dataset, c_j represents the number of sample categories of dataset d_j. To calculate the MPCE, we need to calculate the per-class Error (PCE) firstly. The evaluation indicators are defined as follows.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{38}$$

$$PCE_j = \frac{1 - Accuracy}{c_j} \tag{39}$$

$$MPCE = \frac{1}{M}\sum_{j=1}^{M} PCE_j, j = \{1, 2\cdots, M\} \tag{40}$$

Here, PCE_j represents the detection error rate for per-class of dataset d_j, $_{MPCE}$ represents the Mean of detection error rate for per-class in data pool D, M represents the number of datasets in the data pool D. In this study, $M = 44$. The experimental results are listed in the Table 7.

As can be seen from the Table 7, the classification performance of the proposed method is satisfactory. ALSTM model shows good results on 18 datasets. However, the classification result of ALSTM model is lower than 60 % on 7 datasets. Moreover, MPCE is provided with the characteristic to reflect the overall classification performance of the model. It can be seen from the Table 7, the MPCE of the proposed method is satisfactory. These results mean that the proposed method has prominent classification effect not only on imbalanced datasets but also on balanced datasets.

6. Conclusions

In practical engineering, most time series data sets are in the form of serious inter-class imbalance. That is, the number of normal samples is much more than that of abnormal sample, thus the data set is presence of skewed class distributions and underrepresented for minority class. For this case, many classification algorithms are difficult to learn adequate feature representation due to insufficient representative samples of the minority classes, leading to a serious decline in the detection accuracy for the minority classes. In this study, aiming at the problem that the detection accuracy of the minority class will be inaccurate because of skew data distribution, we present a cost-sensitive hybrid network (CSHN) model for imbalanced time-series data anomaly detection. The novelty is that the CSHN adopts a two-flow structure that contains a CNN and a GRU network, and combining with a cost-sensitive loss function. The proposed CSHN model solves the problem that the existing time series classification methods do not take imbalanced datasets into account. We investigate the performance of CSHN on the practical engineering datasets and the UCR public datasets, results show that our method has prominent detection effect on the imbalanced time series datasets. The proposed method can be used in many domains such as meteorology, telemetry, finance, aerospace measurement & control, medicine, industrial engineering and so on.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant No.62376212; Shaanxi province key research and development project, No.2022GY-087.

References

Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence, 35*, 1798–1828.
Bagnall, A., Lines, J., Hills, J., & Bostrom, A. (2015). Time-series classification with cote: The collective of transformation-based ensembles. *IEEE Transactions on Knowledge and Data Engineering, 27*, 2522–2535.
Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). Smote:Synthetic minority over-sampling technique. *Journal of artificial intelligence research, 16*, 321–357.
Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint arXiv:1412.3555.
Cao, H., Tan, V. Y., & Pang, J. Z. (2014). A parsimonious mixture of gaussian trees model for oversampling in imbalanced and multimodal time-series classification. *IEEE transactions on neural networks and learning systems, 25*, 2226–2239.
Chen, Y., Keogh, E., Hu, B., Begum, N., Bagnall, A., Mueen, A., & Batista,G. (2015).The ucr time series classification archive. www.cs.ucr.edu/~eamonn/time_series_data/.
Cui, Z., Chen, W., & Chen, Y. (2016). Multi-scale convolutional neural networks for time series classification. arXiv preprint arXiv:1603.06995.
Chen, Z., Duan, J., Kang, L., & Qiu, G. (2021). Class-imbalanced deep learning via a class-balanced ensemble. *IEEE Transactions on Neural Networks and Learning Systems, 33*, 5626–5640.
Cheng, X., Shi, F., Liu, X., Zhao, M., & Chen, S. (2021). A novel deep class-imbalanced semisupervised model for wind turbine blade icing detection. *IEEE Transactions on Neural Networks and Learning Systems, 33*, 2558–2579.
Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., & Keogh, E. (2008). Querying and mining of time series data: Experimental comparison of representations and distance measures. Proceedings of the Vldb Endowment.
Dong, Q., Gong, S., & Zhu, X. (2018). Imbalanced deep learning by minority class incremental rectification. *IEEE transactions on pattern analysis and machine intelligence, 41*, 1367–1381.
Fernando, K. R. M., & Tsokos, C. P. (2021). Dynamically weighted balanced loss: Class imbalanced learning and confidence calibration of deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems, 33*, 2940–2951.
Groutage, D., & Bennink, D. (2000). Feature sets for nonstationary signals derived from moments of the singular value decomposition of cohen-posch(positive time-frequency) distributions. *IEEE Transactions on Signal Processing, 48*, 1498–1503.
He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering, 21*, 1263–1284.
He, H. (2011). *Self-adaptive systems for machine intelligence*. John Wiley & Sons.
Hao, P., Zhan, Y., Wang, L., Niu, Z., & Shakir, M. (2015). Feature selection of time series modis data for early crop classification using random forest: A case study in kansas, usa. *Remote Sensing, 7*, 5347–5369.
Https://keras.io/.
Http://scikit-learn.org/.
Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448–456). PMLR.
Jeong, Y.-S., & Jayaraman, R. (2015). Support vector-based algorithms with weighted dynamic time warping kernel function for time series classification. *Knowledge-based systems, 75*, 184–191.
Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
Karim, F., Majumdar, S., Darabi, H., & Chen, S. (2017). Lstm fully convolutional networks for time series classification. *IEEE Access, 6*, 1662–1669.
Khan, S. H., Hayat, M., Bennamoun, M., Sohel, F. A., & Togneri, R. (2017). Cost-sensitive learning of deep feature representations from imbalanced data. *IEEE transactions on neural networks and learning systems, 29*, 3573–3587.
Liu, X.-Y., Wu, J., & Zhou, Z.-H. (2008). Exploratory undersampling for classimbalance learning. IEEE Transactions on Systems, Man, and Cybernetics. *Part B (Cybernetics), 39*, 539–550.
Lecun, Y. A., Bottou, L., Orr, G. B., & Muller, K. R.. (2012). *Efficient BackProp*. Neural Networks: Tricks of the Trade.
Lin, M., Chen, Q., & Yan, S. (2013). Network in network. arXiv preprint arXiv: 1312.4400.
Lines, J., & Bagnall, A. (2015). Time series classification with ensembles of elastic distance measures. *Data Mining and Knowledge Discovery, 29*, 565–592.
Li, H. (2015a). Accurate and efficient classification based on common principal components analysis for multivariate time series. *Neurocomputing, 171*.
Li, H. (2015b). On-line and dynamic time warping for time series data mining. *International Journal of Machine Learning and Cybernetics, 6*, 145–153.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In Proceedings of the IEEE international conference on computer vision (pp. 2980–2988).

Liang, W., Pei, H., Cai, Q., & Wang, Y. (2020). Scalp eeg epileptogenic zone recognition and localization based on long-term recurrent convolutional network. *Neurocomputing, 396*, 569–576.

Mehdiyev, N., Lahann, J., Emrich, A., Enke, D., Fettke, P., & Loos, P. (2017). Time series classification using deep learning for process planning: A case from the process industry. *Procedia Computer Science, 114*, 242–249.

Priyadarsini, K., Hemakesavulu, O., Karthik, S., & Karanam, S. R. (2021). Towards effective time series classification of multi-class imbalanced learning. In *In 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)* (pp. 1576–1582). IEEE.

Rakthanmanon, T., & Keogh, E. (2013). Fast shapelets: A scalable algorithm for discovering time series shapelets. In proceedings of the 2013 SIAM International Conference on Data Mining (pp. 668–676). SIAM.

Rajesh, K. N., & Dhuli, R. (2018). Classification of imbalanced ecg beats using re-sampling techniques and adaboost ensemble classifier. *Biomedical Signal Processing and Control, 41*, 242–254.

Ren, M., Zeng, W., Yang, B., & Urtasun, R. (2018). Proceedings of the 35th international conference on machine learning. PMLR.

Shen, R., Zou, F., Song, J., Yan, K., & Zhou, K. (2018). Efui: An ensemble framework using uncertain inference for pornographic image recognition. *Neurocomputing, 322*, 166–176.

Wang, H., Cui, Z., Chen, Y., Avidan, M., Abdallah, A. B., & Kronzer, A. (2017a). *Cost-sensitive deep learning for early readmission prediction at a major hospital*. Canada Proc. BIOKDD.

Wang, Z., Yan, W., & Oates, T. (2017b). Time series classification from scratch with deep neural networks: A strong baseline. In *In 2017 International joint conference on neural networks (IJCNN)* (pp. 1578–1585). IEEE.

Yang, Q., & Wu, X. (2006). 10 challenging problems in data mining research. *International Journal of Information Technology & Decision Making, 5*, 597–604.

Ye, L., & Keogh, E. (2011). Time series shapelets: A novel technique that allows accurate, interpretable and fast classification. *Data mining and knowledge discovery, 22*, 149–182.

Yang, Z., Chen, W., Wang, F., & Xu, B. (2019). Effectively training neural machine translation models with monolingual data. *Neurocomputing, 333*, 240–247.

Zhou, Z.-H., & Liu, X.-Y. (2005). Training cost-sensitive neural networks with methods addressing the class imbalance problem. *IEEE Transactions on knowledge and data engineering, 18*, 63–77.

Zheng, Y., Liu, Q., Chen, E., Ge, Y., & Zhao, J. L. (2016). Exploiting multichannels deep convolutional neural networks for multivariate time series classification. *Frontiers of Computer Science, 10*, 96–112.

Zhang, C., Tan, K. C., Li, H., & Hong, G. S. (2018). A cost-sensitive deep belief network for imbalanced classification. *IEEE transactions on neural networks and learning systems, 30*, 109–122.